CODE ANALYSIST REPORT

OKTOBER 2026



FUNCTIONAL PROGRAMMING

MODULE 2 - Effect-free Programming and Declarative Paradigm

COMPILED BY:

Rico Shandika Jovial Agista

202410370110433

Pemrograman Fungsional G

1. Pure Function

15 Point

Exercise 1: functional analysis

Complete the code by calling add_bonus2() and run the program multiple times (2–3 times) and observe the changes in both result and exam_scores. Why does add_bonus2() modify the original exam_scores data even though it does not use the global keyword? Explain your analysis. Based on its characteristics, does the add_bonus2() function qualify as a Pure Function or an Impure Function? Explain your reasons.

Run 1

```
#202410370110433
exam_scores = [70, 80, 85]

def add_bonus2(bonus):
    for i in range(len(exam_scores)):
        exam_scores[i] += bonus
    return exam_scores
```

✓ 0.0s

```
#202410370110433
print("Before:", exam_scores)
result = add_bonus2(5)
print("New result:", result)
print("Original exam_scores:", exam_scores)
```

✓ 0.0s

```
Before: [70, 80, 85]
New result: [75, 85, 90]
Original exam_scores: [75, 85, 90]
```

Run 2

```
#202410370110433
exam_scores = [70, 80, 85]

def add_bonus2(bonus):
    for i in range(len(exam_scores)):
        exam_scores[i] += bonus
    return exam_scores
```

[56] ✓ 0.0s

```
#202410370110433
print("Before:", exam_scores)
result = add_bonus2(5)
print("New result:", result)
print("Original exam_scores:", exam_scores)
```

[58] ✓ 0.0s

```
Before: [75, 85, 90]
New result: [80, 90, 95]
Original exam_scores: [80, 90, 95]
```

Run 3

```
#202410370110433
exam_scores = [70, 80, 85]

def add_bonus2(bonus):
    for i in range(len(exam_scores)):
        exam_scores[i] += bonus
    return exam_scores
```

[56] ✓ 0.0s

```
#202410370110433
print("Before:", exam_scores)
result = add_bonus2(5)
print("New result:", result)
print("Original exam_scores:", exam_scores)
```

[59] ✓ 0.0s

```
Before: [80, 90, 95]
New result: [85, 95, 100]
Original exam_scores: [85, 95, 100]
```

Explanations:

add_bonus2() mengubah exam_scores karena variabel itu adalah list (mutable object) dan fungsi mengaksesnya sebagai variabel global tanpa membuat salinan.

Kata kunci global hanya diperlukan untuk meng-assign ulang nama variabel (exam_scores = ...).

Operasi exam_scores[i] += bonus adalah modifikasi in-place pada objek yang sama di

memori, sehingga Python tidak butuh global.

Fungsi mengembalikan exam_scores itu sendiri, jadi result dan exam_scores menunjuk ke objek yang sama.

Setiap kali sel dijalankan ulang, nilai terus bertambah karena data asli ikut berubah

add_bonus2() termasuk Impure Function, karena:

1. Menimbulkan side effect (mengubah data di luar scope fungsi).
2. Hasilnya tidak deterministik: input bonus=5 yang sama menghasilkan output berbeda tiap dipanggil, karena bergantung pada state exam_scores.
3. Bergantung pada variabel eksternal, bukan hanya parameternya.

2. Immutability

15 Point

Exercise 2: Data duplication with shallow and deep copy

```
user = { "name" : "putri", "scores" : [90, 85]}
```

1. Create duplicates of the user variable using both Shallow Copy (store it in a variable named user_shallow) and Deep Copy (store it in a variable named user_deep).
2. In each copied object (user_shallow and user_deep), change the scores value from 90 to 75.
3. Check the memory addresses of the modified index, and compare them with the memory address in the original user variable.

Observe the original data output and its memory address, then explain why the shallow copy and deep copy results are different!

```python
#202410370110433
import copy

user = {"name": "putri", "scores": [90, 85]}

user_shallow = copy.copy(user)
user_deep = copy.deepcopy(user)

# Ubah skor pertama pada kedua salinan
user_shallow["scores"][0] = 75
user_deep["scores"][0] = 75

print("Original user:", user)
print("Shallow copy:", user_shallow)
print("Deep copy:", user_deep)

print("\nID list scores:")
print("user:", id(user["scores"]))
print("user_shallow:", id(user_shallow["scores"]))
print("user_deep:", id(user_deep["scores"]))

print("\nApakah list scores memiliki objek yang sama?")
print("user dan user_shallow:", user["scores"] is user_shallow["scores"])
print("user dan user_deep:", user["scores"] is user_deep["scores"])
```

0.0s

```
Original user: {'name': 'putri', 'scores': [75, 85]}
Shallow copy: {'name': 'putri', 'scores': [75, 85]}
Deep copy: {'name': 'putri', 'scores': [75, 85]}

ID list scores:
user: 2191814345728
user_shallow: 2191814345728
user_deep: 2191815378048

Apakah list scores memiliki objek yang sama?
user dan user_shallow: True
user dan user_deep: False
```

Explanations:

Output yang diamati:

Original user: {'name': 'putri', 'scores': [75, 85]}
Shallow copy : {'name': 'putri', 'scores': [75, 85]}
Deep copy : {'name': 'putri', 'scores': [75, 85]}

user : 2191814345728
user_shallow : 2191814345728 (sama)
user_deep : 2191815378048 (berbeda)

- Shallow copy (copy.copy) hanya membuat dictionary baru di level luar, sedangkan list scores di dalamnya hanya disalin referensinya. user["scores"] dan user_shallow["scores"] menunjuk ke list yang sama (ID sama, is = True). Mengubah user_shallow["scores"][0] ikut mengubah data asli user menjadi 75.

- Deep copy (copy.deepcopy) menyalin secara rekursif sampai level terdalam, sehingga list scores menjadi objek baru dengan ID berbeda (is = False). Perubahan pada user_deep tidak memengaruhi user.

Nilai 75 di user berasal dari modifikasi lewat user_shallow, bukan dari user_deep. Untuk data nested, deep copy diperlukan agar immutability tetap terjaga.

3. Iterator

20 Point

Exercise 3: Using iter() and next() to Process Tuple Data

Create a tuple named sensor_data = (23.5, 24.0, 22.8, 21.1). Convert this tuple into an iterator object using the iter() function. Retrieve and print the first two values from the iterator using the next() function, and then print the remaining data that hasn't been fetched yet using a standard for loop!

1. Observe the output of the for loop. Why does the for loop immediately print the remaining data (the third data point) instead of iterating from the first element?
2. Explain how the for loop mechanism works behind the scenes in Python in handling iterator objects without using the next() function explicitly?

```python
#202410370110433
sensor_data = (23.5, 24.0, 22.8, 21.1)

sensor_iterator = iter(sensor_data)

print("Nilai pertama:", next(sensor_iterator))
print("Nilai kedua:", next(sensor_iterator))

print("Data yang tersisa:")
for value in sensor_iterator:
    print(value)
```

0.0s

```
Nilai pertama: 23.5
Nilai kedua: 24.0
Data yang tersisa:
22.8
21.1
```

Explanations:

1. Kenapa for loop langsung menampilkan data ketiga? iter() mengubah tuple menjadi iterator yang menyimpan posisi saat ini. Dua pemanggilan next() sudah mengambil 23.5 dan 24.0, sehingga pointer berada di elemen ketiga. Iterator hanya bergerak maju dan tidak reset, jadi for loop melanjutkan dari 22.8 lalu 21.1, bukan dari awal.

2. Mekanisme for loop:

   - Python memanggil iter() pada objek. Iterator memang mengembalikan dirinya sendiri (iter(it) is it), sehingga posisinya tidak ter-reset.
   - Loop memanggil next() secara otomatis berulang kali dan menyimpan hasilnya ke variabel loop.

- Saat iterator habis, next() memunculkan StopIteration. For loop menangkap exception ini secara internal dan menghentikan perulangan tanpa error.

4. List Comprehension

15 Point

Exercise 4: Convert to List Comprehension

Convert the following code into a list comprehension:

```
result = []
for letter in "universitas":
    if letter not in "aiueo":
        result.append(letter.upper())

print(result)
```

```
['N', 'V', 'R', 'S', 'T', 'S']
```



```
#202410370110433
word = "universitas"

# menggunakan loop
result = []
for letter in word:
    if letter not in "aiueo":
        result.append(letter.upper())

print("Hasil loop:", result)

# list Comprehension
result_comprehension = [
    letter.upper()
    for letter in word
    if letter not in "aiueo"
]

print("Hasil List Comprehension:", result_comprehension)
print("Tipe objek:", type(result_comprehension))

[3]
Hasil loop: ['N', 'V', 'R', 'S', 'T', 'S']
Hasil List Comprehension: ['N', 'V', 'R', 'S', 'T', 'S']
Tipe objek: <class 'list'>
```

Explanations of Code:

result_comprehension = [letter.upper() for letter in word if letter not in "aiueo"]

- letter.upper() adalah expression: huruf diubah menjadi kapital.
- for letter in word adalah iterable: menelusuri tiap karakter pada "universitas".
- if letter not in "aiueo" adalah condition: hanya huruf konsonan yang

dimasukkan.

Hasilnya ['N', 'V', 'R', 'S', 'T', 'S'], sama dengan versi loop. Versi ini lebih deklaratif (menyatakan apa yang diinginkan, bukan bagaimana membangunnya dengan append) dan lebih ringkas, tanpa list kosong dan mutasi manual.

5. Generator Expression

15 Point

Exercise 5: Create Generator Expression

Create a Generator Expression that produces numbers from 1 to 5, then display its values using next() inside a while loop. Add exception handling to manage the scenario when the data runs out (StopIteration), and print a message indicating that the iterator has finished.

```
#202410370110433
number_generator = (number for number in range(1, 6))

while True:
    try:
        value = next(number_generator)
        print(value)
    except StopIteration:
        print("Iterator telah selesai.")
        break
```

✓ 0.0s

```
1
2
3
4
5
Iterator telah selesai.
```

Explanations of Code:

number_generator = (number for number in range(1, 6))

- Tanda kurung ( ) membuat generator expression, yaitu objek lazy yang menghasilkan 1 sampai 5 satu per satu.
- while True memanggil next(number_generator) pada setiap iterasi untuk mengambil nilai berikutnya.
- Setelah nilai 5 diambil, next() berikutnya memunculkan StopIteration. Blok try-except menangkapnya, mencetak "Iterator telah selesai.", lalu break menghentikan loop.

- Exception handling dibutuhkan karena StopIteration adalah sinyal bahwa data habis. Tanpa except, program akan berhenti dengan error.

6. Generator & Lazy Evaluation

20 Point

Exercise 6: Create Generator

Instructions:

1. Create a generator function named filter_emergency_log(log_list) that accepts the log data.
2. The function must evaluate each log line. If a log line starts with the keyword "ERROR", emit (yield) that log line progressively.
3. In the main program, invoke the generator function and store it in a variable named error_bucket.
4. Manually retrieve the first and second values using the next() function.

If the server log data volume increases to 10,000,000 rows, why is using a generator function with yield much safer for computer memory compared to a regular function that collects all the results into a list and returns them all at once with return?

```python
#202410370110433
raw_logs = [
    "INFO: User login successful",
    "ERROR: Database connection failed",
    "INFO: Page rendered in 25ms",
    "ERROR: Unauthorized access attempt detected",
    "INFO: User logout successful",
]

def filter_emergency_log(log_list):
    for log in log_list:
        if log.startswith("ERROR"):
            yield log

error_bucket = filter_emergency_log(raw_logs)

print("Tipe error_bucket:", type(error_bucket))
print("Log darurat pertama:", next(error_bucket))
print("Log darurat kedua:", next(error_bucket))
```

```
Tipe error_bucket: <class 'generator'>
Log darurat pertama: ERROR: Database connection failed
Log darurat kedua: ERROR: Unauthorized access attempt detected
```

Explanations:
filter_emergency_log() adalah generator function karena memakai yield. error_bucket = filter_emergency_log(raw_logs) belum menjalankan isi fungsi, hanya membuat generator object. Isi fungsi baru berjalan saat next() dipanggil:

- next() pertama menjalankan fungsi sampai yield log ERROR pertama (Database connection failed), lalu fungsi ditangguhkan.

- next() kedua melanjutkan dari baris setelah yield dan berhenti di ERROR kedua (Unauthorized access attempt detected).

Question 4: 10.000.000 baris log

- Fungsi biasa + return list: semua baris ERROR dikumpulkan ke list lalu dikembalikan sekaligus. Seluruh hasil harus tersimpan di RAM sebelum dipakai (eager), dan memori yang dibutuhkan tumbuh sebanding jumlah data, sehingga berisiko kehabisan memori / crash.

- Generator + yield: hanya satu baris yang dihasilkan dan diproses pada satu waktu, lalu fungsi ditangguhkan sambil mempertahankan state lokalnya. Penggunaan memori konstan (hanya instruksi dan state saat ini), berapa pun jumlah barisnya. Ini sejalan dengan hasil getsizeof() di modul: list 800.984 bytes vs generator 200 bytes.

CODE ANALYSIS ASSESSMENT

Code Analysis	Points (Total 20%)
Exercise Implementation	**10**
Seluruh Exercise selesai dikerjakan dengan benar (poin akumulasi dari seluruh exercise yang ada).	100
modifikasi kode tidak berhasil diselesaikan dengan benar	-50% (setiap exercise)
Exercise dikerjakan, namun kode/output salah	-30% (setiap exercise)
Jawaban tidak mengandung identitas praktikan (Nama/NIM)	-20% (setiap exercise)
Selesai melebihi jam praktikum	-100% (setiap exercise)
Practicum Report	**10**

Menggunakan template resmi, seluruh bagian explanation diisi dengan jawaban yang benar dan jelas	100
Jawaban salah atau jawaban kurang tepat	-50% (setiap exercise)
Penjelasan jawaban ditemukan identik dengan praktikan lain	-100% (setiap exercise)
Report dikirim melebihi dateline (saat praktikum)	-40 point (dari nilai akhir)